



Experiment 03: Continuous Integration Pipeline using Jenkins / GitHub Actions

Experiment No.	03
Mapped CO	CO3
Aim	To set up a Continuous Integration (CI) pipeline that automatically builds and tests an application on every code change using GitHub Actions, with Jenkins as an alternative.
Tools / Software	GitHub account; Jenkins (optional); A sample application (Python or Node.js); Web browser / terminal

Theory

Continuous Integration (CI) is the practice of frequently merging developer changes into a shared mainline repository. Each change is automatically verified through a build and test process so that integration errors and defects are detected early. A CI server or cloud CI service checks out the latest source code, installs dependencies, builds the application and runs test cases. The final result is shown as pass or fail feedback to the development team.

GitHub Actions implements CI as code using YAML workflow files stored in the `.github/workflows` directory of the repository. A workflow is triggered by events such as push or pull_request and is made up of jobs and steps that run on a hosted runner. Jenkins can perform the same task through a Pipeline job using a Jenkinsfile and webhooks from GitHub.

Procedure

1. Create or open a GitHub repository containing a sample Python or Node.js application with a test suite.
2. Create the folder `.github/workflows` in the repository and add a workflow file named `ci.yml`.
3. Define workflow triggers for push and pull_request events on the main branch.
4. Add workflow steps to check out the code, set up the runtime, install dependencies and execute tests.
5. Commit and push the workflow file to GitHub.
6. Open the Actions tab in GitHub and observe the workflow run. Confirm that the build and test jobs complete successfully with a green check.
7. As an alternative, create a Jenkins Pipeline job, point it to the repository, configure the GitHub webhook, and use a Jenkinsfile containing build and test stages.

Sample Code / Commands

.github/workflows/ci.yml (Python example)

```
name: CI

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
```



PARSHWANATH CHARITABLE TRUST'S
A.P. SHAH INSTITUTE OF TECHNOLOGY
Department of Computer Science and Engineering
Data Science



```
build-test:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-python@v5
      with:
        python-version: '3.12'
    - run: pip install -r requirements.txt
    - run: pytest -q
```

Jenkinsfile (declarative pipeline alternative)

```
pipeline {
  agent any
  stages {
    stage('Build') {
      steps { sh 'pip install -r requirements.txt' }
    }
    stage('Test') {
      steps { sh 'pytest -q' }
    }
  }
}
```

Observation / Experiment Output

Instructions to students: Paste screenshots or copied outputs in the boxes below after completing CI workflow setup and execution. Keep screenshots clear and readable.

Name	Roll No.	Batch	Date
Pushkar Shinde	58	B3	_____

1. GitHub Repository and Sample Application Output

The screenshot shows a GitHub repository named "DevOps-APSIT" by user "Pushkar3232". The repository is public and has 7 commits. The commit history shows files like .github/workflows, app, docker, k8s, terraform, tests, .gitignore, and README.md. The repository also has a README, activity, stars, watching, and forks section.

2. CI Workflow File / Jenkinsfile Output



PARSHWANATH CHARITABLE TRUST'S
A.P. SHAH INSTITUTE OF TECHNOLOGY
Department of Computer Science and Engineering
Data Science



← CI

✓ ci: add automated build and test pipeline #3

Re-run all jobs

...

Summary

All jobs

build-test

Run details

Usage

Workflow file

Annotations

1 warning

build-test

succeeded 3 minutes ago in 10s

Search logs

Refresh

Settings

> ✓ Set up job

> ✓ Checkout code

> ✓ Set up Python

> ✓ Install dependencies

> ✓ Run tests

> ✓ Post Set up Python

> ✓ Post Checkout code

> ✓ Complete job

1s

1s

0s

4s

0s

1s

0s

0s

3. Pipeline Trigger Output



Workflow file for this run

[.github/workflows/ci.yml](#) at [54c91cc](#)

```
1  name: CI
2
3  on:
4    push:
5      branches: [ main ]
6    pull_request:
7      branches: [ main ]
8
9  jobs:
10   build-test:
11     runs-on: ubuntu-latest
12
13     steps:
14       - name: Checkout code
15         uses: actions/checkout@v4
16
17       - name: Set up Python
18         uses: actions/setup-python@v5
19         with:
20           python-version: "3.12"
21
22       - name: Install dependencies
23         run: pip install -r app/requirements.txt
24
25       - name: Run tests
26         run: pytest -q
```

4. Build and Test Execution Output



PARSHWANATH CHARITABLE TRUST'S
A.P. SHAH INSTITUTE OF TECHNOLOGY
Department of Computer Science and Engineering
Data Science



← □

✓ ci: add automated build and test pipeline #3 Re-run all jobs ...

Summary

All jobs

build-test

Run details

Usage

Workflow file

Annotations

1 warning

build-test

succeeded 4 minutes ago in 10s

Search logs

Set up job 1s

Checkout code 1s

Set up Python 0s

Install dependencies 4s

Run tests 0s

Post Set up Python 1s

Post Checkout code 0s

Complete job 0s

5. Successful CI Status Output

← □

✓ ci: add automated build and test pipeline #3 Re-run all jobs ...

Summary

All jobs

build-test

Run details

Usage

Workflow file

Annotations

1 warning

build-test

succeeded 4 minutes ago in 10s

Search logs

Set up job 1s

Checkout code 1s

Set up Python 0s

Install dependencies 4s

Run tests 0s

Post Set up Python 1s

Post Checkout code 0s

Complete job 0s

GitHub Repository Link: <https://github.com/Pushkar3232/DevOps-APSIT>

GitHub Actions / Jenkins Build Link: <https://github.com/Pushkar3232/DevOps-APSIT/actions>

Final Observation Statement:

The Continuous Integration pipeline was successfully configured using GitHub Actions to automatically build and test the application. The workflow was triggered by a code push and completed successfully with all tests passing



Expected Output

On every push or pull request, the CI workflow should run automatically, install dependencies, execute the test suite and report a pass/fail status in the GitHub Actions tab or Jenkins build history.

Conclusion

An automated Continuous Integration pipeline was configured so that each committed code change triggers a build and test cycle, providing rapid feedback on code quality and application health.

Viva / Review Questions

1. What problem does Continuous Integration solve?
2. What is the role of a webhook in a CI pipeline?
3. Differentiate Continuous Integration, Continuous Delivery and Continuous Deployment.
4. What is a runner in GitHub Actions?

Marks: ____ / 10

Date: _____ **Signature of Faculty:** _____